

Universidad de San Carlos de Guatemala
Centro Universitario de Occidente
División de Ciencias de la Ingeniería
Teoría de Sistemas 1



Manual Técnico

Registro académico: 201831260
Nombre: Michael Kristopher Marín Reyes

Quetzaltenango, Mayo de 2026.

Índice

Tecnologías utilizadas.....	3
Arquitectura del Sistema.....	3
Capas del Sistemas.....	3
Capa de controladores (API REST).....	3
Capa de Lógica de negocio (Services).....	3
Capa de Persistencia (TypeORM).....	3
Seguridad y Autenticación.....	3
Especificaciones de la Base de Datos (PostgreSQL).....	4
Arquitectura del Frontend.....	4
Diseño: Utility-First con Tailwind CSS.....	4
Consistencia visual.....	4
Reducción e CSS personalizado.....	5
Optimización del Bundle (PurgeCSS).....	5
Implementación de Diseño Responsivo y Adaptativo.....	5
Experiencia de Usuario Avanzada.....	5

Manual Técnico

Este manual contiene información sobre la arquitectura utilizada para el desarrollo de SYSHUB, contiene información relevante sobre cómo está desarrollado además de incluir información sobre cómo se conecta tanto Frontend, Backend y Base de datos.

Tecnologías utilizadas

Las tecnologías utilizadas para el desarrollo son:

- NestJS para el desarrollo de la API.
- Vue para el desarrollo del Frontend.
- PostgreSQL para la base de datos.

Arquitectura del Sistema

Como se indicó, el backend está construido sobre el framework NestJS v11, utilizando una arquitectura basada en Clean Architecture y Domain-Driven Design.

Capas del Sistemas

Capa de controladores (API REST)

Éste se encarga de gestionar las peticiones HTTP entrantes, aplica validaciones y transforma respuestas para el cliente.

Capa de Lógica de negocio (Services)

Contiene la inteligencia del sistema. Aquí se gestionan las reglas de negocio y validaciones que permiten el correcto funcionamiento entre datos entre múltiples repositorios, además de cumplir con las necesidades del sistema.

Capa de Persistencia (TypeORM)

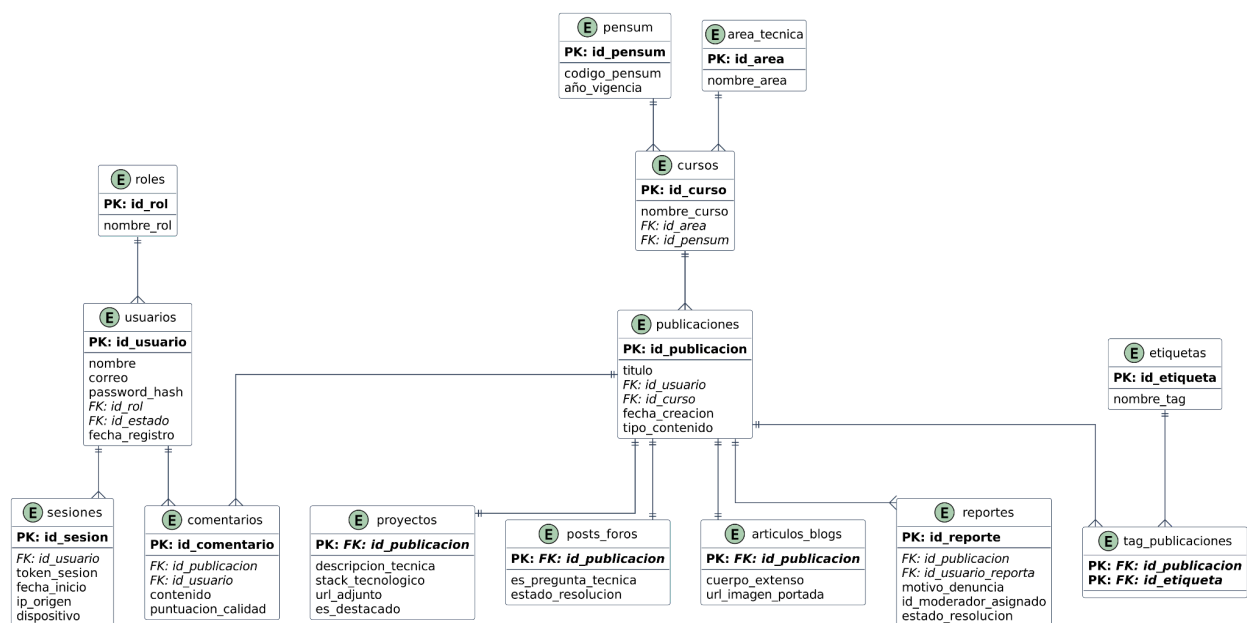
Utiliza un Mapeo Objeto-Relacional (ORM) para interactuar con PostgreSQL de forma segura, evitando inyecciones SQL que es básico en cualquier sistema que se realice, y garantizando la coherencia de tipos entre la base de datos y TypeScript.

Seguridad y Autenticación

Se implementó la estrategia de JWT (JSON Web Tokens) con [Passport.js](#), con esto permite una comunicación estable y segura entre frontend en Vue y el backend.

Especificaciones de la Base de Datos (PostgreSQL)

El diseño de la base de datos Syshub se enfoca en la integridad referencial y la escalabilidad. En este caso a diferencia de la entrega de la fase 1 sólo se agregó una tabla intermedia para gestionar los cursos y su relación con los auxiliares, el diagrama se presenta a continuación.



Arquitectura del Frontend

El frontend sigue un modelo de arquitectura de componentes basado en la reactividad y el diseño responsivo.

Diseño: Utility-First con Tailwind CSS

A diferencia de frameworks tradicionales basados en componentes predefinidos, se utilizó Tailwind CSS para construir una interfaz gráfica totalmente personalizada y optimizada.

Consistencia visual

El uso de escalas predefinidas para espacios, tipografía y colores garantiza que toda la aplicación se mantenga sobre la misma paleta de colores y estética sin importar la complejidad del componente.

Reducción e CSS personalizado

Se minimizó la creación de código css dentro de cada archivo .vue, escribiendo las clases directamente en los archivos .vue lo que facilita el mantenimiento y lectura del código.

Optimización del Bundle (PurgeCSS)

Tailwind elimina automáticamente todo el css que no se utiliza en la aplicación final, resultando en un archivo de estilos extremadamente ligero, lo que acelera la carga inicial del navegador.

Implementación de Diseño Responsivo y Adaptativo

La interfaz de Syshub fue diseñada para funcionar en cualquier dispositivo con acceso a internet, permitiendo así que todos los usuarios consulten información desde cualquier dispositivo.

Experiencia de Usuario Avanzada

Tailwind permitió implementar características modernas que mejoran la interacción.

- Transiciones y estados: uso de clases como hover: y focus: para proporcionar feedback visual inmediato cuando el usuario interactúa con botones, campos de texto y etiquetas.
- Se configuró el archivo [tailwind.config.js](#) para añadir la paleta de colores siguiendo los mockups que se presentaron en la fase 1.